

## Video Transcript

### Video 2: JSON Web Tokens Architecture (04:12)

So, I want to take you through using JSON Web Tokens to authenticate and to authorize. Now, often there'll be two servers. So, this is the authorization server and then, we have the app here that needs to understand if the user is authorized or not. So, the first stage is for the authorization server to receive a login request from the user. So, the user sends an HTTP message and in the body of that message, posts the username and password. The authorization server here then compares that with its list of authorized users and decides whether this person should be given a JSON web token or not. Now, a JSON web token is an encrypted set of data, and it's encrypted with a secret key.

Here, I've called it 'abc'. Now, that secret key is shared with the application server, and it could be many application servers. We might have a lot of web servers, but they would share this secret. Now if, for example, the username is 'John' and the right password is presented, then John will be granted a JSON web token. So, if they're granted a JSON web token, the username and the role that they're allowed will be encrypted with this secret and that is the JSON web token. And that's then sent back to the user. Now, the user needs to present that JSON web token in a request to the app server. The app server uses the shared secret key to decrypt the message and will then get the username and the role. And so, they know it's John, and they know John's role is administrator.

And let's suppose, that that is sufficient, then the app server will respond with the data that they request. Otherwise, the app server will respond with a negative message saying, data unavailable. Okay, so, that's the basic of how that works. Now, this is the encryption process. So, here John and Admin are encrypted with the secret key, some random string, and this is what the JSON web token looks like. It looks like gibberish, but it contains three parts, and we'll describe later what these three parts are. But this is what the user needs to send to the app server, and they need to put this in the header of the HTTP request.

So, this security token is embedded in the HTTP header. This is an example of doing that. We're going to do it in Postman. But if we wanted to write Python code, then this is how we would embed that security token, that JSON web token. We'd put it in the authorization part of the

header, and it would be a bearer token, and then we'd send that HTTP request programmatically to the URL. Here I'm saying localhost:4000/books, but that's what it looks like in code. So, next we'll take a look at how this works in practice on an example problem. Bye for now.

